

Refactoring del Sistema de Notificaciones

Spring Boot · RabbitMQ · PostgreSQL · Observer + Factory Pattern

1. Contexto y objetivo

El sistema usa actualmente un patrón Observer + Factory para crear notificaciones. El problema es que cuando un evento tiene múltiples destinatarios (ej. todos los miembros de un Grupo o Templo al crear un Culto), se persiste una fila UserNotification por persona con el mismo contenido, duplicando datos innecesariamente.

El objetivo es:

- Separar la notificación general (Notification) de la entrega individual (UserNotification).
- Encolar el fanout a RabbitMQ para que el envío se haga de forma asíncrona.
- Personalizar el mensaje que le llega a cada persona (saludo con nombre, datos relevantes) sin duplicar el cuerpo de la notificación.
- Mantener el patrón Observer/Factory ya existente, solo ampliar su responsabilidad.

2. Nuevo modelo de datos

Revisar el proyecto antes de generar migraciones para adaptar enums, nombres de entidades y convenciones existentes.

2.1 Tabla: notification

- id UUID PK
- type ENUM: EVENT | MINISTRY | ADMINISTRATIVE
- context_id UUID nullable — FK al evento/recurso que originó la notificación (solo cuando type = EVENT)
- context_entity_type VARCHAR nullable — GRUPO | TEMPLO (solo cuando type = EVENT)
- title VARCHAR
- body TEXT — cuerpo base de la notificación (puede tener placeholders: {{nombre}}, {{fecha}}, etc.)
- data JSONB — payload libre adicional
- channel ENUM: PUSH | EMAIL | SMS
- template VARCHAR — identificador de plantilla
- variables JSONB — variables para renderizar la plantilla (compartidas; las personales van en UserNotification)
- created_at TIMESTAMP

2.2 Tabla: user_notification

- id UUID PK

- notification_id UUID FK → notification.id
- user_id UUID FK → user.id
- personal_variables JSONB — variables exclusivas del destinatario: {"nombre": "Juan", "cargo": "Díacono"}
- read_at TIMESTAMP nullable — null = no leída
- status ENUM: PENDING | SENT | READ | FAILED
- created_at TIMESTAMP

2.3 Tabla: notification_failed

- id UUID PK
- user_id UUID FK → user.id
- notification_id UUID nullable FK — referencia a la notificación original si existe
- type ENUM (mismo que Notification.type)
- channel ENUM
- personal_variables JSONB
- status ENUM: PENDING_RETRY | EXHAUSTED
- retry_at TIMESTAMP — cuándo se debe reintentar
- retry_count INT DEFAULT 0
- last_error TEXT
- created_at TIMESTAMP

💡 Crear índices en: `user_notification(user_id)`, `user_notification(notification_id)`, `notification_failed(user_id, status)`, `notification_failed(retry_at)`.

3. Flujo general de creación y envío

3.1 Flujo para notificaciones broadcast (Cultos, reuniones, eventos)

1. El servicio de dominio (ej. CultoService) persiste el Culto y emite un evento de dominio CultoCreatedEvent.
2. El Observer (CultoNotificationObserver) recibe el evento e invoca NotificationService.createBroadcast(event, recipients).
3. NotificationService: (a) persiste una sola fila en notification; (b) publica un mensaje en RabbitMQ con el notificationId y la lista de destinatarios (o el criterio de búsqueda: grupold / templold).
4. El Consumer (NotificationFanoutConsumer) recibe el mensaje, obtiene la lista de usuarios, y por cada usuario: (a) crea su UserNotification con personal_variables = {nombre, cargo, etc.}; (b) renderiza el mensaje final combinando Notification.variables + UserNotification.personal_variables; (c) envía por el canal correspondiente (Push/Email/SMS); (d) actualiza status = SENT o registra en notification_failed.

3.2 Flujo para notificaciones personales (cuenta, funciones ministeriales individuales)

5. El Observer correspondiente invoca NotificationService.createPersonal(type, userId, variables).
6. NotificationService persiste Notification + UserNotification directamente, y publica en RabbitMQ un mensaje de envío individual.
7. El Consumer procesa, envía y actualiza status.

4. Configuración de RabbitMQ

4.1 Exchanges y colas

Crear dos exchanges y sus colas correspondientes:

```
Exchange: notifications.broadcast (type: direct | fanout según preferencia)
Queue:    notifications.broadcast.queue

Exchange: notifications.personal
Queue:    notifications.personal.queue

Exchange: notifications.retry      (para reintentos con DLX + TTL)
Queue:    notifications.retry.queue
```

Configurar Dead Letter Exchange (DLX) en la cola de reintentos con TTL escalonado (ej. 1 min, 5 min, 15 min) para reintentos automáticos. Después de N intentos (configurable en application.yml), mover a notification_failed con status = EXHAUSTED.

4.2 Mensaje de broadcast (BroadcastNotificationMessage)

```
notificationId: UUID
contextEntityType: String // GRUPO | TEMPLO
contextId: UUID           // id del grupo o templo
channel: String
templateId: String
baseVariables: Map<String, Object> // variables compartidas
```

4.3 Mensaje de envío individual (PersonalNotificationMessage)

```
notificationId: UUID
userNotificationId: UUID
userId: UUID
channel: String
templateId: String
personalVariables: Map<String, Object> // nombre, cargo, etc.
```

5. Cambios en el patrón Observer + Factory

5.1 Observers existentes que deben actualizarse

El agente debe buscar en el proyecto todas las clases que implementen el Observer de notificaciones (buscar: implements *NotificationObserver, @EventListener con eventos de notificación, o similar). Para cada Observer:

- Si el evento es broadcast (Culto, Reunion, EventoMinisterial grupal): cambiar la llamada a NotificationService.createBroadcast(...) y pasar el criterio de destinatarios (grupold o templold).
- Si el evento es personal (cambio de contraseña, asignación de función ministerial individual, etc.): mantener llamada a NotificationService.createPersonal(...).

5.2 NotificationFactory

Ampliar (no reemplazar) el Factory para que además del objeto Notification construya:

- El BroadcastNotificationMessage listo para encolar, cuando el tipo lo requiera.
- Las personal_variables base del destinatario (el Consumer las complementará con datos del usuario al momento de enviar).

💡 *El Factory NO debe resolver la lista de usuarios. Eso es responsabilidad del Consumer, que consultará la DB al procesar el mensaje para evitar serializar listas grandes en la cola.*

5.3 NotificationService — nuevos métodos requeridos

```
// Para eventos con múltiples destinatarios
Notification createBroadcast(NotificationType type, UUID contextId,
                             ContextEntityType contextEntityType,
                             String templateId, Map<String,Object> variables,
                             String channel)

// Para notificaciones de un solo destinatario
Notification createPersonal(NotificationType type, UUID userId,
                             String templateId, Map<String,Object> variables,
                             String channel)
```

6. Consumer RabbitMQ y renderizado personalizado

6.1 NotificationFanoutConsumer

Este consumer maneja los mensajes broadcast:

- Recibe BroadcastNotificationMessage.
- Consulta la lista de usuarios activos del Grupo o Templo indicado en el mensaje.
- Por cada usuario, combina Notification.variables (compartidas) con personal_variables del usuario ({nombre: user.fullName, cargo: user.ministerialRole, etc.}).
- Renderiza el body y title de la notificación usando el templateId + variables combinadas.
- Crea y persiste UserNotification con status = PENDING.
- Delega el envío real al NotificationSenderService (que internamente usa Push/Email/SMS según channel).

- Actualiza status = SENT. En caso de error: persiste NotificationFailed y publica en la cola de reintentos.

6.2 PersonalNotificationConsumer

Maneja mensajes individuales. Flujo similar pero sin el paso de resolución de lista de usuarios: trabaja directamente con el userNotificationId recibido.

6.3 Renderizado del mensaje personalizado

Implementar un TemplateRenderer (puede ser simple con String.format o Mustache/Thymeleaf si ya existe en el proyecto):

```
// Entrada: "Hola {{nombre}}, el culto {{tituloCulto}} es el {{fecha}}"
// Variables: {nombre: "María", tituloCulto: "Culto de Adoración", fecha: "20/01"}
// Salida: "Hola María, el culto Culto de Adoración es el 20/01"
```

 *El nombre del usuario SIEMPRE viene de personal_variables.nombre, nunca de Notification.variables, para garantizar personalización.*

7. Lógica de reintentos con NotificationFailed

Cuando un envío falla en el Consumer:

- Persistir NotificationFailed con status = PENDING_RETRY, retry_count = 0, last_error = mensaje de excepción.
- Publicar en la cola de reintentos (DLX con TTL).
- Cada reintento incrementa retry_count.
- Al superar el máximo de reintentos (configurable, ej. 3), actualizar status = EXHAUSTED. No volver a encolar.

Agregar un @Scheduled job (NotificationRetryScheduler) que verifique periódicamente notification_failed con status = PENDING_RETRY y retry_at <= now(), para casos donde el mensaje no llegó a encolarse (fallos de red con RabbitMQ).

8. Instrucciones específicas para el agente

Antes de generar cualquier código, el agente debe:

8. Leer el modelo actual de UserNotification y entidades relacionadas para entender los campos existentes y no duplicar ni renombrar lo que ya funciona.
9. Identificar los Observers existentes que crean notificaciones (buscar por implementación del patrón Observer o @EventListener de eventos de dominio).
10. Identificar el Factory actual y sus métodos para extenderlo sin romper lo existente.
11. Verificar si ya existe RabbitMQConfig y adaptar la nueva configuración al estilo ya usado en el proyecto.
12. Identificar los nombres reales de Grupo y Templo en el dominio (pueden llamarse Grupo, Cell, Congregation, etc.).

13. Verificar los canales activos ya implementados (Push/Email/SMS) y sus servicios para no duplicarlos.

Generar en este orden:

- Migración SQL (Flyway/Liquibase) para las 3 tablas nuevas con índices.
- Entidades JPA: Notification, UserNotification (actualizada), NotificationFailed.
- Repositorios Spring Data con los queries necesarios.
- RabbitMQConfig con los exchanges, colas y DLX de reintentos.
- DTOs de mensajes: BroadcastNotificationMessage, PersonalNotificationMessage.
- NotificationService con los métodos createBroadcast y createPersonal.
- Factory actualizado.
- Observers actualizados (broadcast vs personal según tipo de evento).
- NotificationFanoutConsumer y PersonalNotificationConsumer.
- TemplateRenderer.
- NotificationRetryScheduler.

💡 Si el agente encuentra que alguna parte del diseño propuesto entra en conflicto con decisiones de arquitectura ya tomadas en el proyecto, debe documentar el conflicto y proponer alternativas antes de generar el código.